

Milestone 4

1) Product summary

UniMarkt is a purpose-built web application that connects Fulda University students, staff, and faculty in a safe, vibrant peer-to-peer marketplace. By requiring registration with an official “@hs-fulda.de” email and vetting every listing through an admin dashboard, UniMarkt ensures a trusted environment where campus community members can confidently buy, sell, exchange, or offer services—from textbooks and electronics to tutoring sessions and part-time positions.

Leveraging a modern React frontend, Python-powered backend, and AWS-hosted media on Amazon S3, the platform delivers lightning-fast searches and image-rich listings. Users enjoy intuitive filtered searches by category, location or price; paginated browsing; and rich item pages featuring high-resolution photos. A built-in messaging system fosters direct, secure communication.

For administrators and faculty, UniMarkt’s robust role-based controls enable streamlined approval workflows, management, and job posting tools, ensuring high-quality content and alignment with campus needs. Future enhancements include AI-driven recommendations, auction capabilities, and secure payment integration—turning UniMarkt into a one-stop hub for campus commerce and collaboration. By focusing exclusively on the Fulda community, UniMarkt not only simplifies transactions but also strengthens campus connections, promotes sustainability through reuse, and offers a dynamic, user-centric experience that scales as the university grows.

1. Unregistered users can browse and view summaries of approved item, service, and job listings.
2. New users can register with an “@hs-fulda.de” email and secure password.
3. Registered users can log in and out with session timeout management.
4. Users can enable app-based two-factor authentication for enhanced account security.
5. Users can post new item listings with title, description, category, images, location, and price.
6. Users can auto-generate product descriptions using AI directly within the listing form.

7. The system checks form data for harmful or prohibited content and displays a warning before saving.
8. Users can edit their own listings, triggering admin re-approval.
9. Users can delete their own listings, removing them from public view.
10. Users can search listings by keyword, category, location, or price range.
11. Users can filter listings by category, location, or price range and sort by date or price.
12. Users can view detailed item pages with images and full details.
13. Buyers and sellers can exchange secure in-site messages tied to listings.
14. Sellers can mark their items as “sold,” removing them from public listings.
15. Admins can review and approve or reject pending listings before publication.
16. Admins can view and search a paginated list of all registered users.
17. Admins can view and filter a paginated list of all product listings.
18. Admins can approve product listings to make them publicly visible.
19. Admins can reject product listings and provide feedback to users.
20. Admins can permanently delete user accounts that violate policies.
21. Admins can permanently delete product listings.
22. Faculty can create job postings with title, description, contact info, department, and remuneration.
23. Faculty can edit their active job postings, setting edits to “pending” for admin re-approval.
24. Faculty can delete their active job postings, removing them from public view.
25. Faculty can archive their job postings, making them invisible to other users and updating status to “archived.”
26. Users can view and browse active job postings, filter by department, and search by keyword.
27. Users can list services (e.g., training, study help) by entering title, description, category, and pricing.
28. Users can edit their own service listings, setting edits to “pending” for admin re-approval.
29. Users can delete their own service listings, marking them deleted and removing them from public view.
30. Users can view available services, browsing approved listings and filtering by category and price.

31. Users can archive their service listings, making them invisible yet accessible to themselves.
32. Users can unarchive their service listings to restore visibility.
33. Admins can approve service listings, making them visible to the public.

2) Usability test plan

1. Test Objectives

This usability test evaluates two core functionalities of the system:

- **Product Search and Messaging Flow:** We want to determine whether users can efficiently register, log in, search for a product, view details, and initiate contact with the product lister.
- **Product Listing Creation Flow:** We aim to assess how easily a user can create an account, navigate to their profile, and list a new product that appears with a “Pending” status.

The primary goal is to ensure **intuitive user flow**, **ease of navigation**, and **overall satisfaction** with the system's interface. This test focuses on user experience and system behavior during these critical tasks. Measuring satisfaction will help identify UI/UX friction points and areas for improvement.

2. Test Background and Setup

System Setup

Participants will be testing a web-based marketplace application. They will access the system using Chrome (v125+) or Edge (v125+). No installation is required. An internet connection is mandatory.

Starting Point

All testers begin from the **homepage of the system** as new users. No prior login credentials will be provided. They will go through registration as part of the usability flow.

Intended Users

The system is designed for general users with basic computer literacy. Ideal testers are:

- People who buy/sell products online.

- Students or adults aged 18–40.
- Users familiar with common platforms like eBay, Facebook Marketplace, etc.

URL of the System to Be Tested

<http://18.199.222.21:3000/>

What is to Be Measured

For this test, the focus is on **user satisfaction**, not performance metrics. After each task, users will be asked to fill out a **Likert-scale-based satisfaction questionnaire** to assess their experience with:

- Ease of completing the task
- Clarity of interface
- Confidence in completing actions

3. Usability Task Description

Instructions to the Tester

Please follow the two tasks below carefully. After each task, you will be asked to answer a few questions about your experience.

Task 1: Search and Message a Lister

1. Open the provided system URL.
2. Register a new account using any email and password.
3. Log in to the system.
4. Use the search bar or homepage to find a product of interest.
5. Click on any product to view its detail page.
6. Click the "**Send Message**" button to message the lister.
7. Go to the **Messages/Inbox** section to confirm that your message has been sent

Task 2: Create a New Product Listing

1. Log in or register if not already.
2. Navigate to your **Profile** section.
3. Click on "**Create Listing**" or "**Add Product**".
4. Fill in the product title, description, price, and other details.
5. Submit the form.
6. You should be redirected to your profile where the new listing is shown with the status "**Pending**".

After finishing each task, fill in the short Likert-scale questionnaire below.

Measuring Effectiveness

Effectiveness will be measured by **task completion rate**: whether the user was able to complete the required actions without assistance or critical errors.

Measuring Efficiency

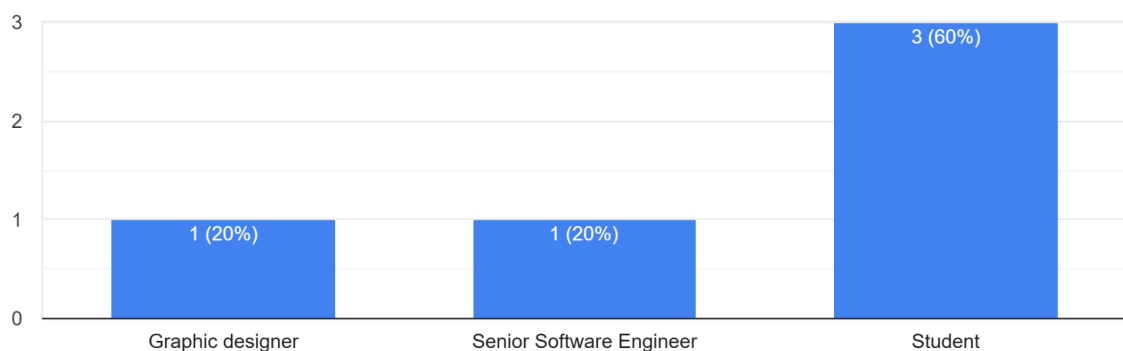
Efficiency will be measured by **time taken to complete each task**, and whether users hesitated, asked questions, or repeated steps during the process.

As part of the testing phase for the UniMarkt, feedback was collected from five individuals to evaluate the app's performance and user experience. The reviewers came from diverse backgrounds, including:

- A male senior software engineer - 30 years old
- A female graphic designer - 25 years old
- A female student - 24 years old
- Two male students - both 26 years old

What is your occupation?

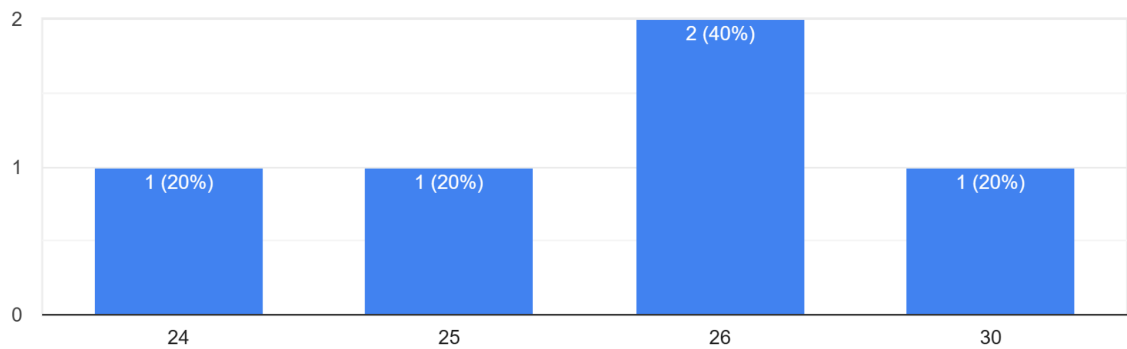
5 responses



Their insights provided valuable perspectives to help identify areas for improvement and enhance the overall quality of the application. The graph below shows the age distribution of the reviewers to give a clear understanding of which age groups they belonged to.

How old are you?

5 responses

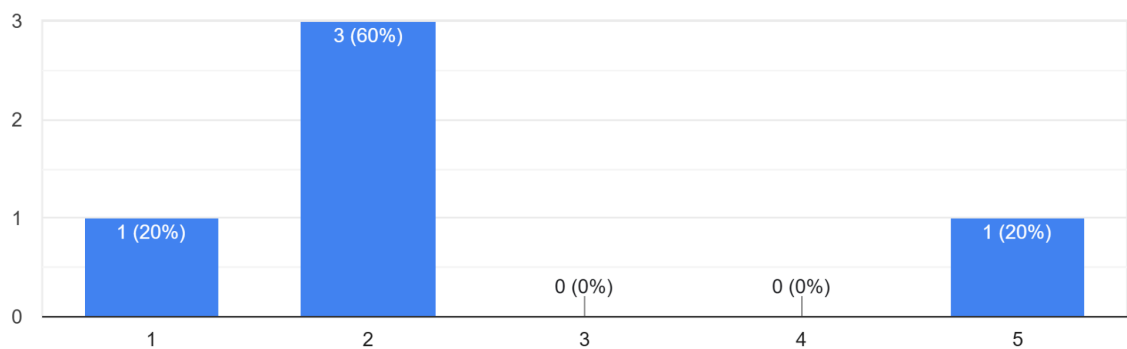


Reviewers were asked to evaluate various statements about the UniMarkt app to assess its usability, design, and overall functionality. They provided their responses using a Likert scale ranging from 1 to 5, where **1 = Strongly Agree** and **5 = Strongly Disagree**.

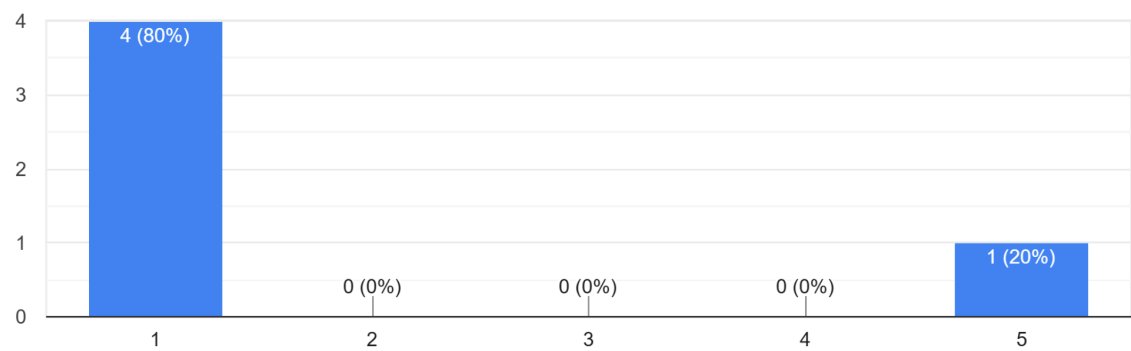
The results of these responses are visualized in the bar graph below, reflecting how well the app met user expectations in different area

The design of the app is user-friendly.

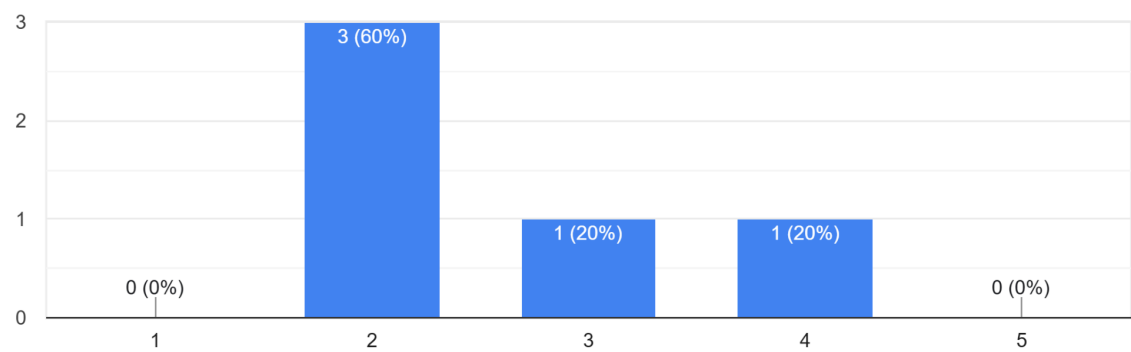
5 responses



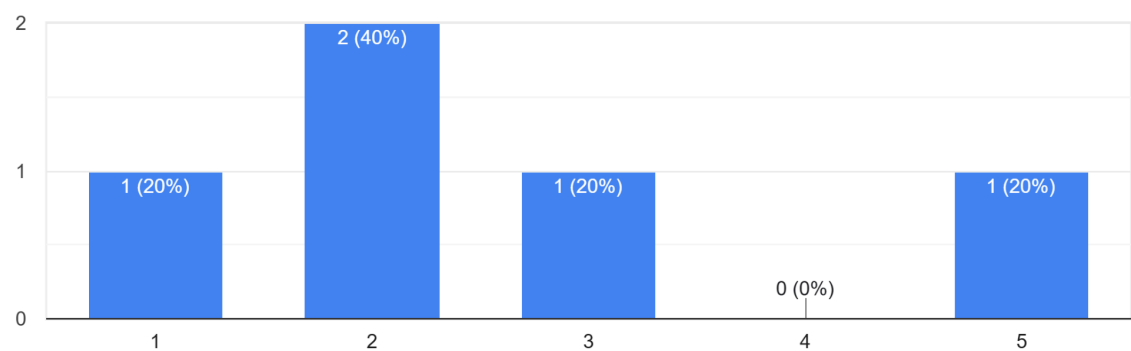
It was easy to register or log in.
5 responses



I understood quickly how to post or search for listings.
5 responses

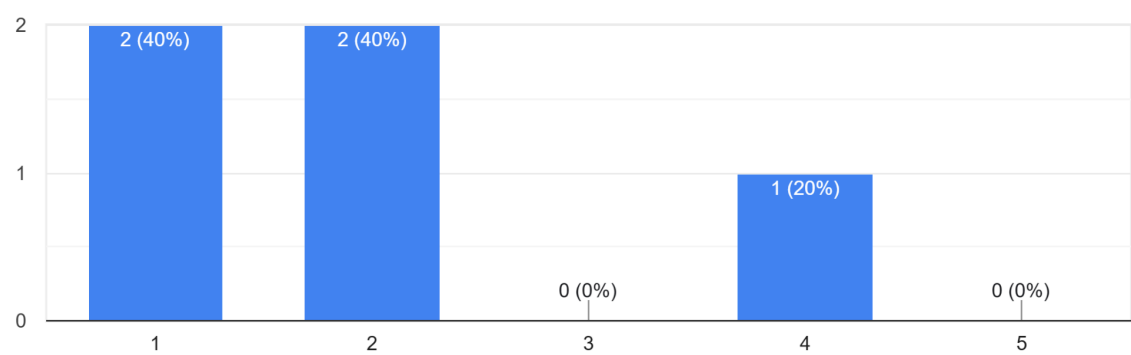


The app looks clean and modern.
5 responses



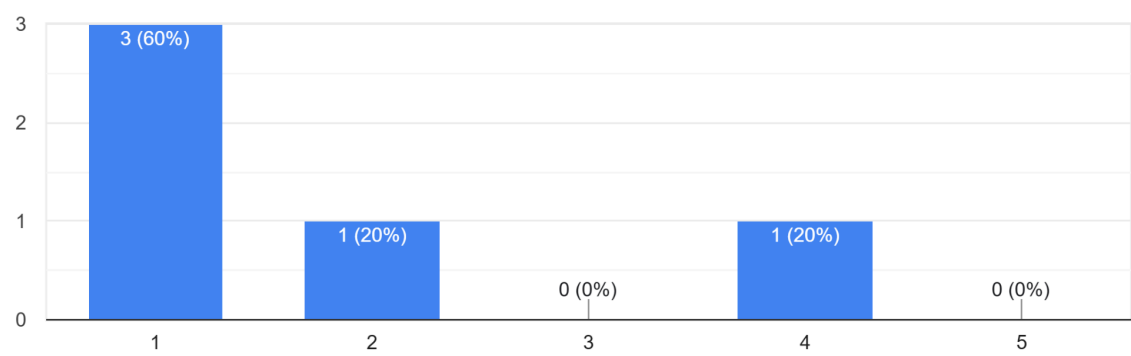
The app is useful for buying and selling within the university.

5 responses



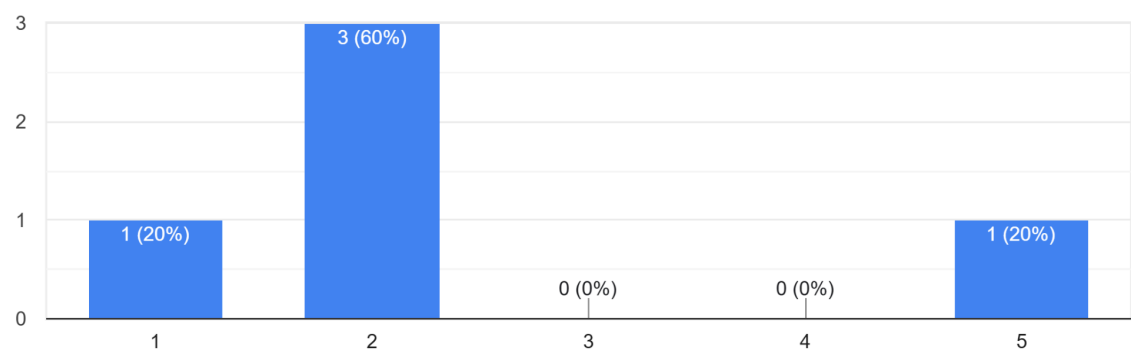
I feel safe and comfortable using this app within the university network.

5 responses



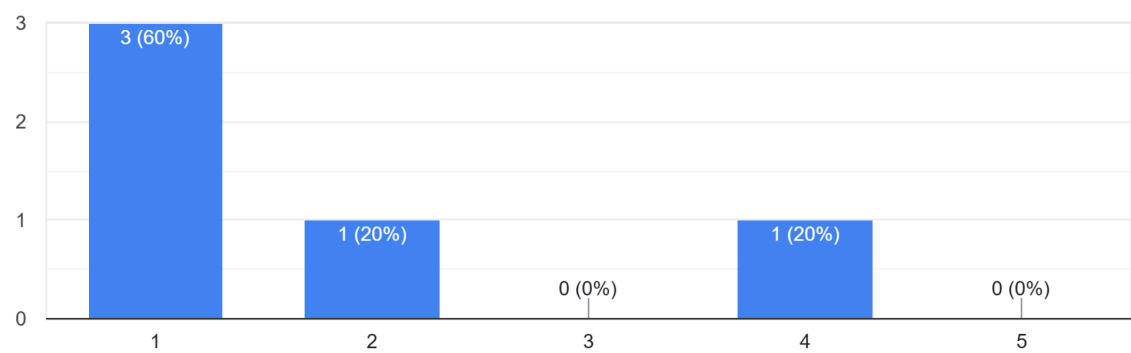
The categories and filters are helpful.

5 responses



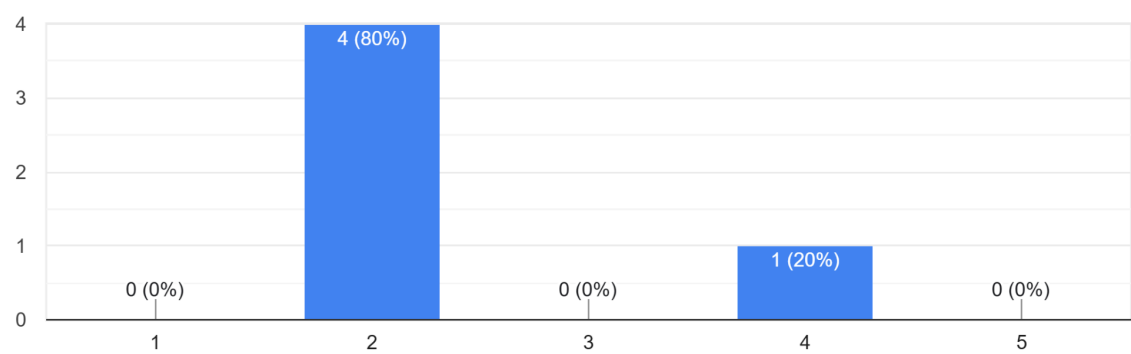
I didn't have to repeat steps due to issues.

5 responses



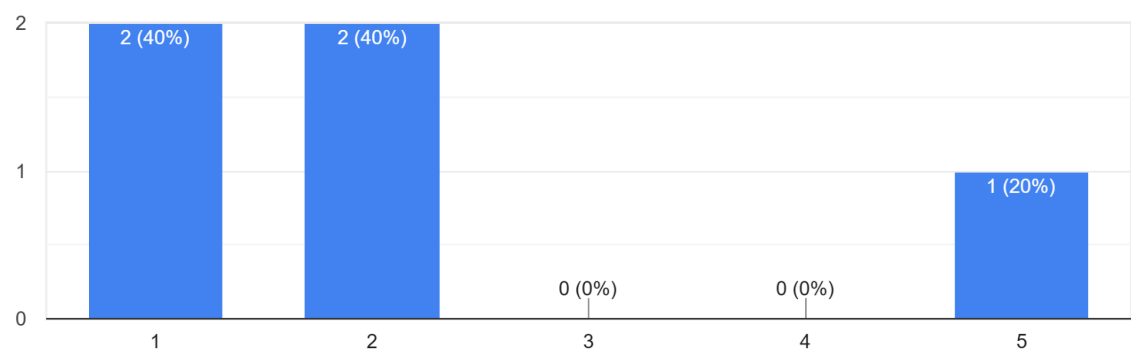
I didn't face any bugs, glitches, or lags.

5 responses



I would use this app regularly if it were fully launched.

5 responses



In addition to the scaled rating questions, several short-answer questions were included to collect more detailed feedback on the app's overall performance and to capture the reviewers' personal experiences and suggestions. The responses to these open-ended questions are summarized below:

What did you like most about the app?

1. The minimalistic design of the website makes it easy to use.
2. The possibility to see job listing and also the skills/service section
3. It looks minimal
4. It's a one place solutions to everything
5. UI looks good and easy to navigate

What did you **NOT** like about the app?

1. Some wordings are unnecessary, for example it's clear from the icon that it is a pickup location so it is repetitive to mention "Pickup Location" in all the cards.
2. It's a bit slow and takes time between different pages
3. It is slow
4. It's sometimes slow
5. It's bit slower

What would you suggest to improve the app?

1. Design should be consistent throughout the application.
2. Some UI changes to make it look better and faster
3. Make some UX improvements and if possible improve the performance
4. The speed
5. Make it faster to load

Any performance issues or technical bugs you noticed?

1. I didn't notice anything special during my usage.
2. not issues but slow

3. The site is slow
4. Sometimes the buttons are working late
5. When I click on something it takes time to load

The user reviews gathered from the testing were mixed. While some areas of the app were considered good and met user expectations, other parts revealed occasional slowdowns and aspects where the process could be made faster or more intuitive. Overall, the feedback highlights strengths in stability and ease of use but also points to opportunities for improving speed and reducing delays in certain tasks. This balanced input will help guide further improvements to enhance the user experience.

3) QA test plan

Create & Search Product

1. Test Objectives

To verify that the product creation and search functionality works correctly in the web application, ensuring:

- Valid and invalid product creation flows are handled
- Search returns correct results for valid terms
- Proper behavior when no search results are found

2. HW and SW Setup

Component	Details
Hardware	PC with 8GB RAM
Operating System	Windows 11
Browsers	Chrome (v125+), Edge (v125+)
Software Stack	Next.js frontend, Playwright

Test URL <http://18.199.90.158:3000/>

3. Feature to be Tested

- Create Product
- Search Product and message
- Homepage Filter UI

4. QA Test Plan Table

Test #	Test Title	Test Description	Test Input	Expected Output	Chrom e	Edge
TC0 1	Homepag e shows filter options	Verify filter section loads on home page	Page load	Filter headings (e.g. "Pickup Location", "Price") visible	PASS	PAS S
TC0 2	Test Product Search	Test product search with a valid query	"coffee"	Product (e.g., Coffee Table) appears in results	PASS	PAS S
TC0 3	Create Product - Valid Input	Create a product with all required fields correctly filled	Valid name, price, category, image	Product appears with "Pending" status	PASS	PAS S

TC0 4	Create Product - Invalid Input	Submit form with missing required fields	Empty name or category	Error message displayed, product not created	PASS	PAS S
TC0 5	Search Product - Valid Term	Search using uppercase/lowerca se mix, and message the sender for buying	"COFFEE"	Same product(s) returned as "coffee", message sent	PASS	PAS S
TC0 6	Product Search - No results	Test search functionality with non-existent product term	"nonexistentterm12 3"	"No results" message or empty list shown	PASS	PAS S

5. Notes

- All tests passed on both browsers (Chrome, Edge) as per screenshot log.
- Test automation was done using Playwright with default timeouts adjusted for better stability.
- No flaky or skipped tests detected.

Q

All6Passed6Failed0Flaky0Skipped0

Project: Chrome7/2/2025, 10:12:45 AMTotal time: 45.6s

▼ example.spec.ts

✓ homepage shows filter options1.9s

example.spec.ts:5

✓ test product search6.3s

example.spec.ts:16

✓ Create Product - Valid Input19.0s

example.spec.ts:45

✓ Create Product - Invalid Input8.6s

example.spec.ts:70

✓ Search Product - Valid Term2.7s

example.spec.ts:88

✓ Search Product - No Results3.6s

example.spec.ts:100

Q

All 6Passed 6Failed 0Flaky 0Skipped 0

Project: Edge

7/2/2025, 10:15:23 AMTotal time: 46.6s

▼ example.spec.ts

✓ homepage shows filter options

example.spec.ts:5

2.5s

✓ test product search

example.spec.ts:16

5.9s

✓ Create Product - Valid Input

example.spec.ts:45

19.8s

✓ Create Product - Invalid Input

example.spec.ts:70

9.8s

✓ Search Product - Valid Term

example.spec.ts:88

2.6s

✓ Search Product - No Results

example.spec.ts:100

2.9s

4) Code Reviews:

Hi Inchara,

I've gone over our `ProductViewSet` implementation (see the file here: <https://github.com/Rohan-Kolagada/GDSDSummer2025/blob/d5fba6234f3de6c5a12096659f3369911c044ae1/Backend/UniMarkt/product/views.py>) and wanted to share some suggestions:

1. Pagination Defaults

We currently hard-code `page\_size = 9` and `max\_page\_size = 100` in our custom paginator.

Proposal: Move these default values into module-level constants (e.g. `DEFAULT\_PAGE\_SIZE = 9`) or configure them in `settings.py` under `REST\_FRAMEWORK['PAGE\_SIZE']`. This way, adjusting pagination site-wide will be much easier.

2. Swagger (drf_yasg)

We're repeating `@swagger\_auto\_schema(tags=["Products"])` on each CRUD method, which clutters the code.

Proposal: Apply `@swagger\_auto\_schema(tags=["Products"])` at the class level on `ProductViewSet`. Then only decorate individual methods when they need extra documentation (like our detailed `list()` parameters).

3. Permissions on Custom Action

Our `archive\_toggle` action currently has `permission\_classes=[]`, allowing any user—even anonymous—to archive products.

Proposal: At minimum, require authenticated users.

Thanks,

Nirajan Khatri

Dear Nirajan,

<https://github.com/Rohan-Kolagada/GDSDSummer2025/blob/44de3d43a550878d2ec9df0d9a03688f3480745c/Frontend/unimark/hooks/useFilters.ts>
Here's a short peer review of the core features and a few issues for useFilters.ts:

Features:

Centralized filter state management: Combines URL state, local state, and derived values for filters, search, category, subcategory, and pagination.
URL synchronization: Uses useQueryState for syncing filter/search params with the URL, enabling shareable/filterable URLs.
Debounced search: Integrates debounced search state to avoid unnecessary API calls.
Clear and reset actions: Includes utility functions for clearing individual or all filters and resetting pagination.
Computed helpers: Exposes booleans like hasAnyFilters, hasFilters, etc., for easy UI logic.
Debug info: Returns debug data for easier troubleshooting and development.

Issues / Suggestions:

Remove unused imports and variables.
Console.log: There's a console.log on line 149, which should be removed in production code.
Inconsistent naming: The return type uses hasAnyFilters, but computed also contains hasFilters, hasPriceFilters, etc., which are not all exposed in the return—consider consistency or exposing all for clarity.
Error typing: The error is cast as Error | null, but the original error type from useProducts might not always be Error. Ensure proper error typing.
Pagination reset: setCurrentPage(1) is called frequently—if filters don't change but set is called, this may cause unnecessary resets. Consider checking if the value actually changed before resetting.
The hook is well-structured, DRY, and covers a wide range of filter-related logic for a product listing page. Just clean up the debug statements and minor typing/naming for even better maintainability.

Thanks,
Murtaza Dhariwala.

Dear Rohan,

I have gone through the MySQL readiness check script. Overall, it functions well and effectively checks for MySQL availability with a retry mechanism.:
https://github.com/Rohan-Kolagada/GDSDSummer2025/blob/dev/Backend/UniMark/check_mysql.py

That said, I do have a few suggestions that could improve the code's robustness and maintainability:

Consider using a context manager (with statement) when creating the socket to ensure it closes properly, even if an error occurs.

Switching to the logging module instead of print() would offer better control over output and is more suitable for production environments.

Making the host, port, and retry interval configurable (e.g., via environment variables or command-line arguments) would enhance reusability.

Including exception messages in the logs can make troubleshooting easier if connection issues arise.

Let me know if you'd like help implementing any of these changes or if you'd like me to refactor the code accordingly.

Best regards,
Inchara Rao

Hey Azeem,

Hope you're doing well! I took a look at your forgot password component and wanted to share some thoughts.<https://github.com/Rohan-Kolagada/GDSDSummer2025/blob/fbce139cad8aa64a34bd6c4020ee0e13811b407e/Frontend/unimark/modules/auth/ui/views/forgot-password-view.tsx>

First off, I really like what you've built here. The component is clean and well-structured, you've done a great job with the form validation using Zod and React Hook Form, and the responsive design looks solid on both mobile and desktop. Nice work!

I noticed a few small things that could make it even better:

There are some unused imports and variables hanging around that you might want to clean up. Also, there's a tiny typo in the heading - "Retreive" should be "Retrieve".

The form submission - right now it's missing error handling and there's no loading state to show users something's happening. Plus, the disabled prop is hardcoded to false, so the form submits regardless of validation state.

It would be great to add some success feedback too, so users know their request went through.

These are all pretty straightforward fixes that would make the user experience much smoother.
Let me know if you want to chat about any of these suggestions or if you'd like me to help implement them.

Regards,
Rohan Kolagada

Hey Murtaza,

Hope you're doing well! I took a look at your messaging-view component and wanted to share some thoughts. [<https://github.com/Rohan-Kolagada/GDSDSummer2025/blob/fbce139cad8aa64a34bd6c4020ee0e13811b407e/Frontend/unimartk/modules/auth/ui/views/forgot-password-view.tsx>] [<https://github.com/Rohan-Kolagada/GDSDSummer2025/blob/dev/Frontend/unimartk/modules/messaging/ui/views/messaging-view.tsx>]

First off, really nice work on this component! It's clear that you've put effort into organizing your logic—your use of TanStack Query for fetching messages and the socket setup for real-time updates is well thought out. The integration of React hooks like useState, useEffect, and useRef is clean and readable, and the layout using Tailwind CSS is neat and responsive.

Here are a few simple suggestions to improve the component.

The socket connection is created but not cleaned up, so adding cleanup in useEffect would help avoid memory leaks. When messages load, there's no visible loading sign; adding a spinner or loader would help users know something is happening. Although errors are detected, there's no message shown to users if something goes wrong—displaying a simple error message would be better.

The chat scrolls to the bottom after sending a message, but it should also do this when new messages arrive so users always see the latest updates. Finally, the send button allows messages with just spaces; disabling the button when the input is empty or trimming input live would prevent empty messages from being sent.

Overall, you've done a great job, and the component is well organized. With a few small improvements to make it easier to use and more reliable, it will work even better.

5) Self-check on best practices for security – ½ page

Major Assets Being Protected:

1. User Data (credentials, profiles)
2. Media Files (stored in Amazon S3)
3. Admin Panel and Application Logic

Major Threats per Asset:

1. User Data – Risks include unauthorized access, credential theft, and brute-force attacks.
2. Media Files – Exposed S3 URLs can lead to unauthorized access and data leaks.
3. Admin Panel – Vulnerable to unauthorized access, code injection, or privilege misuse.

Protection Measures:

- User Data: Passwords are securely stored using Django's built-in password hashing (PBKDF2 by default).
- Media Files: Stored in Amazon S3. Access is managed via direct URLs; bucket permissions reviewed to ensure access is limited.

- Admin Panel: Access is restricted to superusers. Protected by Django's CSRF protection and built-in authentication.

Password Encryption: Confirmed. Passwords are hashed in the database using Django's default password hasher (PBKDF2 + SHA256).

6) Self-check: Adherence to original Non-functional specs

1. The application shall be developed, tested, and deployed using tools and servers approved by the Class CTO and as agreed in Milestone 0. Application delivery shall be from the chosen cloud server. - ON TRACK
2. The application shall be optimized for standard desktop and laptop browsers and must render correctly on the two latest versions of two major browsers. - ON TRACK
3. All or selected application functions shall render well on mobile devices. - ON TRACK
4. Data shall be stored in the database on the team's deployment cloud server. - ON TRACK
5. Full resolution free media shall be downloadable directly, and full resolution media for selling shall be obtained after contacting the seller or owner. - ON TRACK
6. No more than fifty concurrent users shall be accessing the application at any time. - ON TRACK
7. User privacy shall be protected, and all privacy policies shall be appropriately communicated to users. - ON TRACK
8. The language used shall be English, with no localization needed. - ON TRACK
9. The application shall be very easy to use and intuitive. - ON TRACK
10. The application shall follow established architecture patterns. - DONE
11. Application code and its repository shall be easy to inspect and maintain. - DONE
12. Google Analytics shall be used if desired (optional for Fulda teams). - ON TRACK
13. No e-mail clients shall be allowed. - DONE
14. Pay functionality, if any, shall not be implemented or simulated in the user interface. .
DONE
15. Site security basic best practices, as covered in class, shall be applied for main data items. - DONE

16. The application shall be media rich, supporting standard formats for images and video as used in the market today. - DONE
17. Modern software engineering processes and practices, including collaborative and continuous development, shall be used. - DONE
18. For code development, management, and documentation, each team shall use their own GitHub repository set up during Milestone 0. - DONE
19. The application UI for web and mobile shall prominently display the exact text “Fulda University of Applied Sciences Software Engineering Project, Summer 2025 For Demonstration Only” on all pages. - DONE